

# PL/pgSQL: Funzioni e Trigger

Laboratorio di Basi di Dati

May 20, 2026

## Section 1

### Ambiente di Lavoro

- 1 Far partire e mostrare lo status del server su cui gira il DBMS

```
sudo systemctl start postgresql  
sudo systemctl status postgresql
```

- 2 Collegamento alla CLI di postgresql con l'utente "postgres" (utente creato di default con permessi di root sul server)

```
sudo -u postgres psql  
\l
```

- 3 Crea l'utente con una password (e modifica password)

```
CREATE USER dbman WITH PASSWORD 'ciao';
```

oppure:

```
ALTER USER dbman WITH PASSWORD 'nuova_password_sicura';  
\du
```

- 4 Crea un database per questo utente

```
CREATE DATABASE biblioteca;  
\l
```

- 5 Assegna i permessi (privilegi) all'utente su quel database

```
ALTER DATABASE biblioteca OWNER TO dbman;
```

- 6 Esci: \q

**DBeaver** è un client grafico per database. Per connettersi a PostgreSQL:

- ① *Database → New Database Connection*
- ② Selezionare **PostgreSQL**
- ③ Compilare i campi:
  - **Host:** localhost
  - **Port:** 5432
  - **Database:** biblioteca
  - **Username:** dbman
  - **Password:** ciao
- ④ Cliccare **Test Connection**, poi **Finish**  
*Se viene richiesto di scaricare il driver JDBC, accettare.*

Aprire il **SQL Editor** (F3 o tasto destro sulla connessione → *SQL Editor*):

| Azione                   | Scorciatoia  |
|--------------------------|--------------|
| Esegui la query corrente | Ctrl+Enter   |
| Esegui tutto il file     | Alt+X        |
| Formatta il codice SQL   | Ctrl+Shift+F |
| Commento/decommento riga | Ctrl+/<br>   |

Il pannello dei risultati compare in basso dopo l'esecuzione.

Per visualizzare le tabelle create: espandere nel *Database Navigator* biblioteca → Schemas → public → Tables

Per aggiornare la vista dopo modifiche da altri client: **F5**.

Per connettersi tramite terminale (alternativa testuale a DBeaver):

```
psql -h localhost -U dbman -d biblioteca
```

Meta-comandi psql utili:

| Comando         | Effetto                  |
|-----------------|--------------------------|
| \l              | Elenca i DB attivi       |
| \dt             | Elenca le tabelle        |
| \d nome_tabella | Struttura di una tabella |
| \q              | Esci                     |

## Section 2

### Il Problema

Una biblioteca vuole gestire digitalmente i propri servizi. Il sistema deve tenere traccia di:

- **Libri** — titolo, ISBN, anno, numero di copie
- **Autori** — un libro ha un solo autore principale
- **Utenti** — iscritti con dati anagrafici
- **Prestiti** — chi ha preso cosa, quando, e se ha restituito



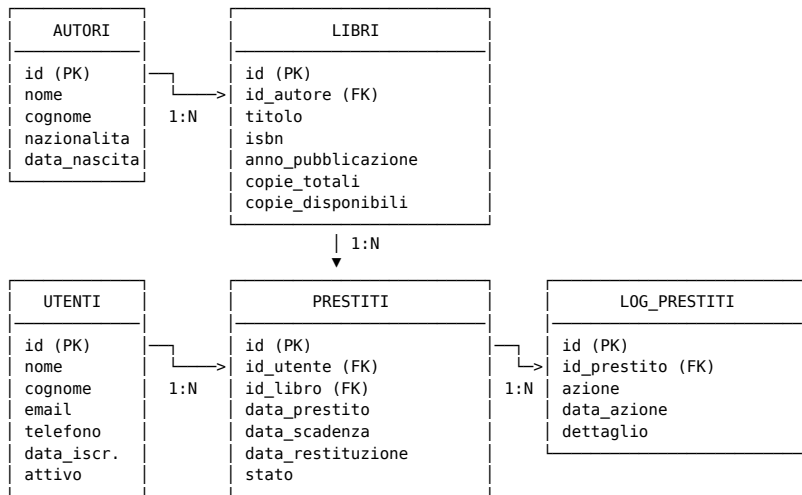
Queste regole guidano la progettazione e le funzioni che scriveremo:

- ❶ Non si può prendere un libro se non ci sono copie disponibili
- ❷ Quando si apre un prestito, `copie_disponibili` decresce di 1
- ❸ Quando si chiude (restituzione), `copie_disponibili` aumenta di 1
- ❹ Un utente non può avere più di **3 prestiti aperti** contemporaneamente
- ❺ Solo gli utenti **attivi** possono effettuare prestiti.
- ❻ Le operazioni sui prestiti devono essere **tracciate** in un log

## Section 3

### Schema Entità-Relazione

# Diagramma ER



## Section 4

### Creazione delle Tabelle

Le tabelle con chiavi esterne vanno create **dopo** quelle a cui puntano.

## **Ordine corretto:**

autori → libri → utenti → prestiti → log\_prestiti

Creare le tabelle nell'ordine sbagliato genera un errore di violazione di chiave esterna.

# Tabella: autori

```
CREATE TABLE autori (  
  id          SERIAL PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  cognome     VARCHAR(100) NOT NULL,  
  nazionalita VARCHAR(100),  
  data_nascita DATE  
);
```

- SERIAL — intero auto-incrementante (1, 2, 3, ...)
- PRIMARY KEY — univoco e NOT NULL, identifica la riga
- NOT NULL — il campo è obbligatorio
- DATE — solo la data, formato YYYY-MM-DD

# Tabella: libri

```
CREATE TABLE libri (  
    id SERIAL PRIMARY KEY,  
    titolo VARCHAR(255) NOT NULL,  
    isbn VARCHAR(20) UNIQUE,  
    anno_pubblicazione INTEGER,  
    copie_totali INTEGER NOT NULL DEFAULT 1  
        CHECK (copie_totali >= 1),  
    copie_disponibili INTEGER NOT NULL DEFAULT 1  
        CHECK (copie_disponibili >= 0),  
    id_autore INTEGER REFERENCES autori(id)  
        ON DELETE SET NULL,  
    CONSTRAINT chk_copie  
        CHECK (copie_disponibili <= copie_totali)  
);
```

- CHECK — vincolo che deve essere soddisfatto per ogni riga
- ON DELETE SET NULL — se l'autore viene cancellato, il campo id\_autore diventa NULL (il libro resta)
- CONSTRAINT chk\_copie — le copie disponibili non possono superare le totali

# Tabella: utenti

```
CREATE TABLE utenti (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  cognome VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  telefono VARCHAR(20),  
  data_iscrizione DATE NOT NULL DEFAULT CURRENT_DATE,  
  attivo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

- DEFAULT CURRENT\_DATE — se non specificata, usa la data di oggi
- BOOLEAN — TRUE / FALSE
- UNIQUE su email — nessun duplicato ammesso



# Tabella: prestiti e log\_prestiti

```
CREATE TABLE prestiti (  
    id SERIAL PRIMARY KEY,  
    id_utente INTEGER NOT NULL REFERENCES utenti(id),  
    id_libro INTEGER NOT NULL REFERENCES libri(id),  
    data_prestito TIMESTAMP NOT NULL DEFAULT NOW(),  
    data_scadenza DATE NOT NULL,  
    data_restituzione TIMESTAMP,  
    stato VARCHAR(20) NOT NULL DEFAULT 'aperto'  
    CHECK (stato IN ('aperto', 'chiuso', 'scaduto'))  
);  
  
CREATE TABLE log_prestiti (  
    id SERIAL PRIMARY KEY,  
    id_prestito INTEGER REFERENCES prestiti(id) ON DELETE SET NULL,  
    azione VARCHAR(50) NOT NULL,  
    data_azione TIMESTAMP NOT NULL DEFAULT NOW(),  
    dettaglio TEXT  
);
```

## Section 5

### Popolamento del Database

# Inserimento dati: autori

```
INSERT INTO autori (nome, cognome, nazionalita, data_nascita) VALUES
('Umberto', 'Eco', 'Italiana', '1932-01-05'),
('George', 'Orwell', 'Britannica', '1903-06-25'),
('Franz', 'Kafka', 'Boema', '1883-07-03'),
('Haruki', 'Murakami', 'Giapponese', '1949-01-12'),
('Elena', 'Ferrante', 'Italiana', NULL);
```

Verificare:

```
SELECT * FROM autori;
```

# Inserimento dati: libri

```
INSERT INTO libri
(titolo, isbn, anno_pubblicazione,
 copie_totali, copie_disponibili, id_autore)
VALUES
('Il Nome della Rosa', '9788845276002', 1980, 3, 3,
 (SELECT id FROM autori WHERE cognome = 'Eco')),
('1984', '9788804668237', 1949, 4, 4,
 (SELECT id FROM autori WHERE cognome = 'Orwell')),
('La Metamorfosi', '9788807900464', 1915, 2, 2,
 (SELECT id FROM autori WHERE cognome = 'Kafka')),
('Norwegian Wood', '9780099520160', 1987, 2, 2,
 (SELECT id FROM autori WHERE cognome = 'Murakami')),
('L'amica geniale', '9788866321590', 2011, 3, 3,
 (SELECT id FROM autori WHERE cognome = 'Ferrante'));
```

*La subquery (SELECT id FROM autori WHERE ...) evita di dover ricordare gli id numerici (ATTENZIONE! Cosa stiamo assumendo?).*

# Inserimento dati: utenti

```
INSERT INTO utenti (nome, cognome, email, attivo) VALUES
('Marco', 'Rossi', 'marco.rossi@email.it', TRUE),
('Giulia', 'Bianchi', 'giulia.bianchi@email.it', TRUE),
('Luca', 'Verdi', 'luca.verdi@email.it', TRUE),
('Chiara', 'Conti', 'chiara.conti@email.it', TRUE),
('Paolo', 'Romano', 'paolo.romano@email.it', FALSE);
```

Verificare:

```
SELECT * FROM utenti;
```

# Inserimento dati: prestiti

```
-- Prestiti già chiusi
INSERT INTO prestiti
    (id_utente, id_libro, data_prestito,
     data_scadenza, data_restituzione, stato)
VALUES
    (1, 2, '2024-01-10', '2024-01-24', '2024-01-22', 'chiuso'),
    (2, 1, '2024-02-05', '2024-02-19', '2024-02-18', 'chiuso');

-- Prestiti ancora aperti
INSERT INTO prestiti (id_utente, id_libro, data_scadenza, stato)
VALUES
    (1, 4, CURRENT_DATE + 9, 'aperto'),
    (2, 3, CURRENT_DATE - 1, 'aperto'),
    (3, 5, CURRENT_DATE - 3, 'scaduto');

-- Aggiorniamo le copie per i prestiti aperti/scaduti
UPDATE libri SET copie_disponibili = copie_disponibili - 1
WHERE id IN (3, 4, 5);
```

## Section 6

### Funzioni in PL/pgSQL

# Cos'è una funzione in PostgreSQL?

Una funzione è un blocco di codice **riutilizzabile** che:

- Accetta parametri in ingresso
- Esegue operazioni (calcoli, query, aggiornamenti)
- Restituisce un valore

PostgreSQL supporta funzioni in **SQL puro** (semplici) e in **PL/pgSQL** (con variabili, condizioni, cicli).

```
CREATE OR REPLACE FUNCTION nome(parametro TIPO)
RETURNS TIPO_RITORNO
LANGUAGE plpgsql
AS $$
DECLARE
    variabile TIPO;
BEGIN
    -- logica
    RETURN valore;
END;
$$;
```



## Funzione 1 — SQL pura: età di un autore

```
CREATE OR REPLACE FUNCTION eta_autore(p_id INTEGER)
RETURNS INTEGER
LANGUAGE sql
AS $$
    SELECT DATE_PART('year', AGE(data_nascita))
    FROM autori
    WHERE id = p_id;
$$;
```

- AGE(data) calcola l'intervallo tra la data e oggi
- DATE\_PART('year', ...) estrae gli anni dall'intervallo

```
-- Test:
SELECT cognome, data_nascita, eta_autore(id) AS eta
FROM autori
ORDER BY cognome;
```

## Funzione 1.1 — PL/pgSQL: età di un autore

```
CREATE OR REPLACE FUNCTION eta_autore_plpgsql(p_id INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_data_nascita DATE;
    v_eta          INTEGER;
BEGIN
    SELECT data_nascita INTO v_data_nascita
    FROM autori
    WHERE id = p_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Autore % non trovato.', p_id;
        RETURN NULL;
    END IF;

    IF v_data_nascita IS NULL THEN
        RAISE NOTICE 'Autore % non ha data di nascita.', p_id;
        RETURN NULL;
    END IF;

    v_eta := DATE_PART('year', AGE(v_data_nascita))::INTEGER;
    RETURN v_eta;
END;
$$;

-- Test:
SELECT cognome, data_nascita, eta_autore_plpgsql(id) AS eta
FROM autori
ORDER BY cognome;

SELECT eta_autore_plpgsql(10)
```

## Funzione 2 — PL/pgSQL: copie disponibili

```
CREATE OR REPLACE FUNCTION conta_copie(p_titolo VARCHAR)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_copie INTEGER;
BEGIN
    SELECT copie_disponibili INTO v_copie
    FROM libri
    WHERE LOWER(titolo) = LOWER(p_titolo);

    IF NOT FOUND THEN
        RAISE NOTICE 'Libro non trovato: %', p_titolo;
        RETURN NULL;
    END IF;

    RETURN v_copie;
END;
$$;
```

- SELECT ... INTO variabile — assegna il risultato a una variabile
- FOUND — variabile speciale: TRUE se la SELECT INTO ha trovato righe
- LOWER() — confronto case-insensitive

```
SELECT conta_copie('1984');
SELECT conta_copie('Libro inesistente');

-- ATTENZIONE! Quale assunzione stiamo facendo?
```

## Funzione 3 — PL/pgSQL: registra un prestito (1/2)

```
CREATE OR REPLACE FUNCTION registra_prestito(  
    p_id_utente INTEGER,  
    p_id_libro  INTEGER,  
    p_giorni    INTEGER DEFAULT 14  
)  
RETURNS INTEGER  
LANGUAGE plpgsql AS $$  
DECLARE  
    v_copie  INTEGER;  
    v_aperti INTEGER;  
    v_id     INTEGER;  
    v_titolo VARCHAR(255);  
BEGIN  
    -- recupero il titolo  
    SELECT titolo INTO v_titolo FROM libri WHERE id = p_id_libro;  
    -- chiamo la funzione precedente  
    v_copie := conta_copie(v_titolo);  
  
    IF v_copie IS NULL THEN  
        RAISE EXCEPTION 'Errore nel recupero delle copie per il libro %', p_id_libro;  
    END IF;  
  
    IF v_copie <= 0 THEN  
        RAISE EXCEPTION 'Nessuna copia disponibile.';  
    END IF;
```

## Funzione 3 — PL/pgSQL: registra un prestito (2/2)

```
SELECT COUNT(*) INTO v_aperti FROM prestiti
WHERE id_utente = p_id_utente
  AND stato IN ('aperto', 'scaduto');
IF v_aperti >= 3 THEN
  RAISE EXCEPTION 'Limite di 3 prestiti aperti raggiunto.';
END IF;

INSERT INTO prestiti (id_utente, id_libro, data_scadenza)
VALUES (p_id_utente, p_id_libro, CURRENT_DATE + p_giorni)
RETURNING id INTO v_id;

UPDATE libri SET copie_disponibili = copie_disponibili - 1
WHERE id = p_id_libro;

RETURN v_id;
END;
$;
```

RETURNING id INTO v\_id recupera il valore di id appena generato dall'INSERT e lo salva nella variabile locale.

## Funzione 3 — Test

```
-- Luca prende "Il Nome della Rosa" (14 giorni, default)
SELECT * FROM prestiti;
SELECT registra_prestito(3, 1);

-- Giulia prende "Norwegian Wood" per 21 giorni
SELECT registra_prestito(2, 4, 21);

-- Verifica:
SELECT p.id, u.cognome, l.titolo, p.data_scadenza, p.stato
FROM prestiti p
JOIN utenti u ON p.id_utente = u.id
JOIN libri l ON p.id_libro = l.id
ORDER BY p.id DESC
LIMIT 5;
```

## Funzione 4 — PL/pgSQL: restituire un libro (1/2)

```
CREATE OR REPLACE FUNCTION restituisci_libro(p_id_prestito INTEGER)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_id_libro INTEGER;
    v_stato    VARCHAR(20);
BEGIN
    SELECT id_libro, stato INTO v_id_libro, v_stato
    FROM prestiti WHERE id = p_id_prestito;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Prestito % non trovato.', p_id_prestito;
    END IF;
    IF v_stato = 'chiuso' THEN
        RAISE EXCEPTION 'Il prestito % è già chiuso.', p_id_prestito;
    END IF;
```

```
UPDATE prestiti
SET stato = 'chiuso', data_restituzione = NOW()
WHERE id = p_id_prestito;

UPDATE libri
SET copie_disponibili = copie_disponibili + 1
WHERE id = v_id_libro;

RETURN 'Prestito ' || p_id_prestito || ' chiuso con successo.';
END;
$;
```

```
-- Test:
SELECT * FROM libri;
SELECT * FROM prestiti;
SELECT restituisci_libro(3);
SELECT id, stato, data_restituzione FROM prestiti WHERE id = 3;
```



## Funzione 5 — PL/pgSQL: gestione scadenza con LOOP (1/2)

```
CREATE OR REPLACE FUNCTION gestione_scadenze()
RETURNS INTEGER LANGUAGE plpgsql AS $$
DECLARE
    r_prestito RECORD;
    v_contatore INTEGER := 0;
BEGIN
    FOR r_prestito IN
        SELECT p.id, p.id_utente, u.nome
        FROM prestiti p
        JOIN utenti u ON p.id_utente = u.id
        WHERE p.stato = 'aperto' AND p.data_scadenza < CURRENT_DATE
    LOOP
        -- 1. Aggiorniamo il record
        UPDATE prestiti SET stato = 'scaduto' WHERE id = r_prestito.id;

        -- 2. Logghiamo l'evento con un dettaglio testuale
        INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
        VALUES (r_prestito.id, 'NOTIFICA_SCADENZA', 'Notifica scadenza per ' || r_prestito.nome);

        v_contatore := v_contatore + 1;
    END LOOP;

    RETURN v_contatore;
END;
$;
```

## Funzione 5 — PL/pgSQL: gestione scadenza con LOOP (2/2)

Chiamiamo la funzione e verifichiamone il comportamento.

```
-- Test:
SELECT gestione_scadenze();

-- Oppure, esecuzione Blocco Anonimo (Script):
DO $$
DECLARE
    v_numero_libri INTEGER;
BEGIN
    -- Eseguiamo la funzione e salviamo il risultato in una variabile locale
    v_numero_libri := gestione_scadenze();

    IF v_numero_libri > 0 THEN
        RAISE NOTICE 'Attenzione: % libri sono passati allo stato SCADUTO.', v_numero_libri;
    ELSE
        RAISE NOTICE 'Non c'erano libri in ritardo oggi.';
    END IF;
END $$;

-- 1. Controlla i prestiti cambiati
SELECT * FROM prestiti WHERE stato = 'scaduto' ORDER BY data_scadenza DESC;
-- 2. Controlla i log generati dal ciclo LOOP
SELECT * FROM log_prestiti WHERE azione = 'NOTIFICA_SCADENZA' ORDER BY data_azione DESC;
```

## Section 7

### Trigger in PL/pgSQL

# Cos'è un Trigger?

Un **trigger** è una procedura eseguita **automaticamente** da PostgreSQL quando avviene un evento su una tabella.

Si compone di **due parti**:

- 1 **La funzione trigger** — contiene la logica, restituisce TRIGGER
- 2 **Il trigger** — collega l'evento alla funzione

-- Parte 1: funzione

```
CREATE OR REPLACE FUNCTION mia_funzione()  
RETURNS TRIGGER LANGUAGE plpgsql  
AS $$  
DECLARE  
    variabile TIP0;  
BEGIN  
    -- NEW = nuova riga (INSERT / UPDATE)  
    -- OLD = vecchia riga (UPDATE / DELETE)  
    RETURN NEW;  
END;  
$$;
```

-- Parte 2: trigger

```
CREATE TRIGGER mio_trigger  
    BEFORE INSERT ON mia_tabella  
    FOR EACH ROW  
    EXECUTE FUNCTION mia_funzione();
```

## 2. Classificazione TRIGGER per Evento

Un trigger si attiva quando viene eseguito uno dei seguenti comandi:

- **INSERT**: Attivato quando nuove righe vengono aggiunte.
- **UPDATE**: Attivato quando righe esistenti vengono modificate.
- **DELETE**: Attivato quando righe vengono rimosse.
- **TRUNCATE**: Attivato quando la tabella viene svuotata (solo per trigger a livello di istruzione).

### 3. Classificazione TRIGGER per Timing

Determina se la logica viene eseguita prima o dopo la modifica dei dati.

| Tipo          | Descrizione                     | Uso Comune                                                  |
|---------------|---------------------------------|-------------------------------------------------------------|
| <b>BEFORE</b> | Eseguito prima dell'operazione. | Validazione dati, modifica automatica dei valori (NEW).     |
| <b>AFTER</b>  | Eseguito dopo l'operazione.     | Logging (audit), aggiornamento di altre tabelle, notifiche. |

## 4. Livello di Esecuzione (Granularità)

Definisce quante volte deve girare la funzione durante una query.

### FOR EACH ROW (Livello Riga)

- Viene eseguito **una volta per ogni riga** colpita dalla query SQL.
- Se un UPDATE modifica 50 righe, il trigger gira 50 volte.
- Permette di accedere alle variabili speciali OLD e NEW.

### FOR EACH STATEMENT (Livello Istruzione)

- Viene eseguito **una sola volta per query**, indipendentemente dal numero di righe coinvolte.
- Ideale per log generici (“L’utente X ha cancellato dei record dalla tabella Y”).
- Più performante se non serve analizzare i dati riga per riga.

All'interno della funzione trigger, PostgreSQL mette a disposizione variabili predefinite:

- **NEW**: Record contenente i dati per INSERT o i nuovi dati per UPDATE. (In DELETE è NULL).
- **OLD**: Record contenente i dati originali prima di UPDATE o DELETE. (In INSERT è NULL).
- **TG\_OP**: Variabile testuale che indica l'operazione (INSERT, UPDATE, DELETE, TRUNCATE).
- **TG\_TABLE\_NAME**: Nome della tabella che ha scatenato il trigger.



## Trigger 1 — BEFORE INSERT: normalizza i dati utente

```
CREATE OR REPLACE FUNCTION trg_normalizza_utente()  
RETURNS TRIGGER LANGUAGE plpgsql AS $$  
BEGIN  
    NEW.email := LOWER(TRIM(NEW.email));  
    NEW.nome := INITCAP(TRIM(NEW.nome));  
    NEW.cognome := INITCAP(TRIM(NEW.cognome));  
    RETURN NEW;  
END;  
$$;
```

```
CREATE TRIGGER trg_before_insert_utenti  
    BEFORE INSERT ON utenti  
    FOR EACH ROW  
    EXECUTE FUNCTION trg_normalizza_utente();
```

- LOWER → minuscolo | TRIM → rimuove spazi | INITCAP → Prima Lettera Maiuscola
- Il trigger modifica NEW **prima** che la riga venga scritta nel database

```
-- Test:  
INSERT INTO utenti (nome, cognome, email, attivo)  
VALUES (' FRANCESCA ', ' de luca ',  
        ' Francesca.DeLuca@EMAIL.IT ', TRUE);  
SELECT nome, cognome, email FROM utenti WHERE cognome = 'De Luca';
```

## Trigger 2 — AFTER INSERT: log apertura prestito

```
CREATE OR REPLACE FUNCTION trg_log_apertura()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_utente TEXT;
    v_libro TEXT;
BEGIN
    SELECT nome || ' ' || cognome INTO v_utente
    FROM utenti WHERE id = NEW.id_utente;

    SELECT titolo INTO v_libro
    FROM libri WHERE id = NEW.id_libro;

    INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
    VALUES (NEW.id, 'APERTURA',
            'Utente: ' || v_utente ||
            ' - Libro: ' || v_libro || '');
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_after_insert_prestiti
AFTER INSERT ON prestiti
FOR EACH ROW
EXECUTE FUNCTION trg_log_apertura();

-- Test:
SELECT registra_prestito(4, 2);
SELECT * FROM log_prestiti ORDER BY id DESC LIMIT 3;
```

## Trigger 3 — AFTER UPDATE: log cambio stato prestito

```
CREATE OR REPLACE FUNCTION trg_log_cambio_stato()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.stato <> NEW.stato THEN
        INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
        VALUES (
            NEW.id,
            'CAMBIO STATO',
            'Stato:' || OLD.stato || ' → ' || NEW.stato
        );
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_after_update_prestiti
AFTER UPDATE ON prestiti
FOR EACH ROW
EXECUTE FUNCTION trg_log_cambio_stato();

-- Test:
SELECT restituisci_libro(4);
SELECT * FROM log_prestiti ORDER BY id DESC LIMIT 3;
```

In un trigger AFTER UPDATE, OLD contiene i valori **prima** della modifica e NEW quelli **dopo**. Il confronto OLD.stato <> NEW.stato permette di reagire solo ai cambiamenti effettivi.

## Trigger 2+3 con TG\_OP - Logging completo (1/2)

```
CREATE OR REPLACE FUNCTION trg_monitoraggio_prestiti()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_info_extra TEXT;
BEGIN
    -- Gestione INSERT (Apertura Prestito)
    IF (TG_OP = 'INSERT') THEN
        INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
        VALUES (NEW.id, 'APERTURA', 'Prestito creato per il libro ID: ' || NEW.id_libro);

    -- Gestione UPDATE (Cambio Stato o Restituzione)
    ELIF (TG_OP = 'UPDATE') THEN
        IF OLD.stato <> NEW.stato THEN
            INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
            VALUES (NEW.id, 'VARIAZIONE', 'Stato passato da ' || OLD.stato || ' a ' || NEW.stato);
        END IF;

    -- Gestione DELETE (Cancellazione record)
    ELIF (TG_OP = 'DELETE') THEN
        -- Nota: Usiamo OLD perché NEW è NULL in una DELETE
        INSERT INTO log_prestiti (id_prestito, azione, dettaglio)
        VALUES (OLD.id, 'CANCELLAZIONE', 'Il record del prestito è stato rimosso dal sistema');
    END IF;

    -- Ricorda: nei trigger di riga devi SEMPRE ritornare NEW (o OLD in caso di delete)
    IF (TG_OP = 'DELETE') THEN RETURN OLD; END IF;
    RETURN NEW;
END;
$;
```

## Trigger 2+3 con TG\_OP - Logging completo (2/2)

```
-- Rimuove i due trigger precedenti dalla tabella prestiti
DROP TRIGGER IF EXISTS trg_after_insert_prestiti ON prestiti;
DROP TRIGGER IF EXISTS trg_after_update_prestiti ON prestiti;

CREATE TRIGGER trg_ciclo_vita_prestiti
  AFTER INSERT OR UPDATE OR DELETE ON prestiti
  FOR EACH ROW
  EXECUTE FUNCTION trg_monitoraggio_prestiti();

select * from prestiti;
select * from log_prestiti;
select * from libri;
SELECT registra_prestito(1, 1);
SELECT restituisci_libro(9);

-- Verifica il log
SELECT * FROM log_prestiti
WHERE azione = 'APERTURA'
ORDER BY id DESC LIMIT 1;
```

## Trigger 4 — BEFORE UPDATE: impedisce copie negative

```
CREATE OR REPLACE FUNCTION trg_controlla_copie()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.copie_disponibili < 0 THEN
        RAISE EXCEPTION
            'Copie disponibili non possono essere negative '
            '(libro: "%").', NEW.titolo;
    END IF;
    IF NEW.copie_disponibili > NEW.copie_totali THEN
        RAISE EXCEPTION
            'Copie disponibili (%) non possono superare '
            'le totali (%) per "%".',
            NEW.copie_disponibili, NEW.copie_totali, NEW.titolo;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_before_update_libri
BEFORE UPDATE ON libri
FOR EACH ROW
EXECUTE FUNCTION trg_controlla_copie();

-- Test (deve fallire):
UPDATE libri SET copie_disponibili = -1 WHERE titolo = '1984';
-- Test (deve riuscire):
UPDATE libri SET copie_disponibili = 2 WHERE titolo = '1984';

-- Cosa succederebbe se eliminassi il trigger e rifacessi il primo update?
```

## Trigger 5 — BEFORE INSERT: blocca utenti non attivi

```
CREATE OR REPLACE FUNCTION trg_check_utente_attivo()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_attivo BOOLEAN;
BEGIN
    SELECT attivo INTO v_attivo
    FROM utenti WHERE id = NEW.id_utente;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Utente % non trovato.', NEW.id_utente;
    END IF;

    IF NOT v_attivo THEN
        RAISE EXCEPTION
            'Utente % non attivo: impossibile creare il prestito.',
            NEW.id_utente;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_before_insert_check_utente
BEFORE INSERT ON prestiti
FOR EACH ROW
EXECUTE FUNCTION trg_check_utente_attivo();

-- Test (deve fallire – Paolo ha attivo = FALSE):
SELECT registra_prestito(5, 1);
```

## Trigger 6 AFTER INSERT su prestiti (1/2)

Quando viene creato un nuovo prestito con stato aperto o scaduto, il database deve decrementare automaticamente copie\_disponibili.

```
CREATE OR REPLACE FUNCTION trg_decrementa_copie_insert()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.stato IN ('aperto', 'scaduto') THEN

        UPDATE libri
        SET copie_disponibili = copie_disponibili - 1
        WHERE id = NEW.id_libro;

    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_after_insert_decrementa_copie
AFTER INSERT ON prestiti
FOR EACH ROW
EXECUTE FUNCTION trg_decrementa_copie_insert();

-- ATTENZIONE! Come gestire la funzione "registra_prestito()?"
```



## Trigger 6 AFTER INSERT su prestiti (2/2)

*-- Test, Verifica copie iniziali*

```
SELECT titolo, copie_disponibili  
FROM libri  
WHERE id = 1;
```

*-- Creazione nuovo prestito*

```
INSERT INTO prestiti  
    (id_utente, id_libro, data_scadenza, stato)  
VALUES  
    (1, 1, CURRENT_DATE + 14, 'aperto');
```

*-- Verifica decremento automatico*

```
SELECT titolo, copie_disponibili  
FROM libri  
WHERE id = 1;
```

## Trigger 7 — AFTER UPDATE su prestiti (1/2)

Quando un prestito cambia stato:

da aperto/scaduto → chiuso → le copie aumentano, da chiuso → aperto/scaduto → le copie diminuiscono

```
CREATE OR REPLACE FUNCTION trg_gestisci_copie_update()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    -- Prestito chiuso: restituzione libro
    IF OLD.stato IN ('aperto', 'scaduto')
    AND NEW.stato = 'chiuso' THEN

        UPDATE libri
        SET copie_disponibili = copie_disponibili + 1
        WHERE id = NEW.id_libro;

    END IF;

    -- Prestito riaperto
    IF OLD.stato = 'chiuso'
    AND NEW.stato IN ('aperto', 'scaduto') THEN

        UPDATE libri
        SET copie_disponibili = copie_disponibili - 1
        WHERE id = NEW.id_libro;

    END IF;

    RETURN NEW;
END;
$;
```

## Trigger 7 — AFTER UPDATE su prestiti (2/2)

```
CREATE TRIGGER trg_after_update_gestisci_copie
  AFTER UPDATE ON prestiti
  FOR EACH ROW
  EXECUTE FUNCTION trg_gestisci_copie_update();

-- Test, Stato iniziale libro
SELECT titolo, copie_disponibili
FROM libri
WHERE id = 1;

-- Chiusura prestito
UPDATE prestiti
SET stato = 'chiuso',
    data_restituzione = NOW()
WHERE id = 6;

-- Verifica incremento copie
SELECT titolo, copie_disponibili
FROM libri
WHERE id = 1;

-- ATTENZIONE! Come gestire la funzione "restituisce_libro()??
```

## Trigger 8 - FOR EACH STATEMENT su libri

```
CREATE OR REPLACE FUNCTION trg_protezione_massiva_libri()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    -- impedisce cancellazioni tra le 08:00 e le 20:00
    IF CURRENT_TIME BETWEEN '08:00:00' AND '20:00:00' THEN
        RAISE EXCEPTION 'Operazioni di manutenzione (DELETE) vietate durante l''orario di apertura.'
    END IF;

    RETURN NULL; -- Nei trigger STATEMENT il valore di ritorno è ignorato
END;
$$;

CREATE OR REPLACE TRIGGER trg_statement_delete_libri
BEFORE DELETE OR TRUNCATE ON libri
FOR EACH STATEMENT
EXECUTE FUNCTION trg_protezione_massiva_libri();

-- Test. Solleva eccezione
DELETE FROM libri;
-- Solleva eccezione
TRUNCATE TABLE libri RESTART IDENTITY CASCADE;
```

```
SELECT
    id,
    id_prestito,
    azione,
    data_azione,
    dettaglio
FROM log_prestiti
ORDER BY id DESC
LIMIT 10;
```